KIROVAULT

File Preview & Thumbnail Engine

A modular, demand-driven preview pipeline with clear format support, cache invalidation and safe fallbacks.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Preview routing matrix

• Images JPG/PNG/GIF/BMP/WebP: thumbnail, zoom, fit and dimensions; animation may initially show first frame.

• Video MP4/MOV/MKV/WebM/AVI: poster frame, duration/resolution and playback only when container and codec are supported.

• PDF: page-one thumbnail, on-demand page rendering, page navigation and text search if supported.

• Text/code/Markdown/JSON/XML/YAML/logs: read-only escaped text, optional syntax highlighting and load-more.

• Audio: playback and metadata where supported; waveform can be later.

• ZIP: list archive members and sizes without extracting; unknown types get metadata + Open with.

Pipeline

• List files and metadata without decoding all content.

• Request thumbnails only for visible/near-visible cards; prioritize selected item.

• Cache lookup uses canonical path and change markers such as size/modification time; optional content hash for stronger invalidation.

• On miss, run bounded background decoder; cache result and update only affected UI item.

• Invalidate when path, size, modified time, decoder or generator version changes.

Resource and security policy

• Never execute scripts, macros or active HTML during preview.

• Escape untrusted text; block remote resource loads by default.

• Cap preview bytes/lines, decoding memory, worker time and concurrency.

• Do not extract archives by default; defend against path traversal if extraction is later added.

• Encrypted/corrupt/unsupported files show stable error state and retry/open-with action.

Cache management

• Keep preview cache separate from source files.

• Use size/age budgets and provide clear-cache control with visible footprint.

• PDF pages render on demand; avoid rendering every page pre-emptively.

• Prefer transient memory cache for sensitive text contents.

Test corpus

• Large and transparent images, odd aspect ratios, corrupted images.

• Videos with varied codecs, variable frame rate and missing metadata.

• Multi-page, encrypted and damaged PDFs.

• Huge text files, alternate encodings, long JSON lines.

• Nested ZIP files and unsafe member paths.

References

• PDF.js: https://mozilla.github.io/pdf.js/

• FFmpeg documentation: https://ffmpeg.org/documentation.html

• FFmpeg legal/license notes: https://ffmpeg.org/legal.html

• Windows Shell thumbnails: https://learn.microsoft.com/en-us/windows/win32/shell/thumbnail-providers

• MDN media formats: https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.